EXP NO: 8	Image Generation Using Variational Autoencoder (Vae)

AIM:

To generate new images using a Variational Autoencoder (VAE) trained on the CIFAR-10 dataset.

ALGORITHM:

1. Load CIFAR-10 and use the first 5,000 images (batch size 128).
2. Build the Encoder: three Conv2d layers (3 - 32 - 64 - 128 channels) followed by two linear layers that give the mean (mu) and log-variance (logvar) of a 64-dimensional latent vector.
3. Apply the reparameterization trick: z = mu + eps * std, where std = exp(0.5 * logvar) and eps is random noise. Build
4. the Decoder: a linear layer and three ConvTranspose2d layers with a Sigmoid output that reconstructs a 3 x 32 x 32 image from z.
5. Define the VAE loss as reconstruction loss (MSE, sum) plus the KL-divergence term.
6. Train the VAE for 5 epochs with the Adam optimizer (learning rate 0.001) and print the average loss per epoch. Sample 16 random latent vectors from a normal distribution, decode them and display the generated images in a
7. grid.

CODE:

```
# EXPERIMENT 8
# Image Generation using Variational Autoencoder (VAE)

import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision
import torchvision.transforms as transforms

import matplotlib.pyplot as plt

from torch.utils.data import DataLoader, Subset

# Device
device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print("Using device:", device)

# Parameters
image_size = 32
batch_size = 128
latent_dim = 64
epochs = 5
learning_rate = 0.001

# Dataset
transform = transforms.Compose([
    transforms.ToTensor()
])

dataset = torchvision.datasets.CIFAR10(
    root="./data",
    train=True,
```

```
    download=True,
    transform=transform
)

# Use small subset for faster Colab execution dataset
= Subset(dataset, range(5000))

dataloader = DataLoader(
    dataset,
    batch_size=batch_size,
    shuffle=True
)

print("Dataset loaded successfully!") #
Encoder
class Encoder(nn.Module):


    def __init__(self, latent_dim):

        super().__init__()

        self.encoder = nn.Sequential(

            nn.Conv2d(
            3, 32,
            kernel_size=4,
            stride=2,
            padding=1
            ),

            nn.ReLU(),

            nn.Conv2d(
                32,             64,
                kernel_size=4,
                stride=2,
                padding=1
            ),

            nn.ReLU(),

            nn.Conv2d(
                64,            128,
                kernel_size=4,
                stride=2,
                padding=1
            ),

            nn.ReLU()
        )

        self.fc_mu = nn.Linear(
            128 * 4 * 4,
            latent_dim
        )

        self.fc_logvar = nn.Linear(
            128 * 4 * 4,
            latent_dim
        )

    def forward(self, x):
```

```
        x = self.encoder(x)

        x = x.view(
            x.size(0),
            -1
        )

        mu = self.fc_mu(x)

        logvar = self.fc_logvar(x)

        return mu, logvar

# Decoder
class Decoder(nn.Module):

    def __init__(self, latent_dim):

        super().__init__()

        self.fc = nn.Linear(
            latent_dim,
            128 * 4 * 4
        )

        self.decoder = nn.Sequential(

            nn.ConvTranspose2d(
                128,            64,
                kernel_size=4,
                stride=2,
                padding=1
            ),

            nn.ReLU(),

            nn.ConvTranspose2d(
                64,             32,
                kernel_size=4,
                stride=2,
                padding=1
            ),

            nn.ReLU(),

            nn.ConvTranspose2d(
                32,             3,
                kernel_size=4,
                stride=2,
                padding=1
            ),

            nn.Sigmoid()
        )

    def forward(self, z):

        x = self.fc(z)

        x = x.view(
            x.size(0),
            128,
```

```
                4,
                4
        )

# VAE     return self.decoder(x)
class VAE(nn.Module):



    def __init__(self, latent_dim):

        super().__init__()

        self.encoder     =     Encoder(latent_dim)

        self.decoder = Decoder(latent_dim)

    def reparameterize(self, mu, logvar):

            std = torch.exp(
                0.5 * logvar
        )

        eps = torch.randn_like(std)

        return mu + eps * std

    def forward(self, x):

        mu, logvar = self.encoder(x)

        z = self.reparameterize(
            mu,
            logvar
        )

        reconstructed = self.decoder(z)

        return reconstructed, mu, logvar

# Loss function
def vae_loss(
    reconstructed,
    original,
    mu,
    logvar
):

    reconstruction_loss = F.mse_loss(
        reconstructed,
        original,
        reduction="sum"
    )

    kl_loss = -0.5 * torch.sum(
        1 + logvar
        - mu.pow(2)
        - logvar.exp()
    )

    return reconstruction_loss + kl_loss

# Create model
```

```
model = VAE(latent_dim).to(device)

optimizer = torch.optim.Adam(
    model.parameters(),
    lr=learning_rate
)

# Training
for epoch in range(epochs):

    model.train()

    total_loss = 0

    for images, _ in dataloader:

        images = images.to(device)

        reconstructed, mu, logvar = model(images)

        loss = vae_loss(
            reconstructed,
            images,
            mu,
            logvar
        )

        optimizer.zero_grad()

        loss.backward()

        optimizer.step()

        total_loss += loss.item()

    average_loss = (
            total_loss /
        len(dataset)
    )

    print(
        f"Epoch {epoch + 1}/{epochs} "
        f"- Loss: {average_loss:.4f}"
    )

# Generate new images
model.eval()

with torch.no_grad():

    z = torch.randn(
        16,
        latent_dim
    ).to(device)

    generated_images = model.decoder(z)

    generated_images = generated_images.cpu()

# Display generated images
grid = torchvision.utils.make_grid(
    generated_images,
    nrow=4
```

```
)

plt.figure(figsize=(8, 8))

plt.imshow(
      grid.permute(1, 2, 0)
)

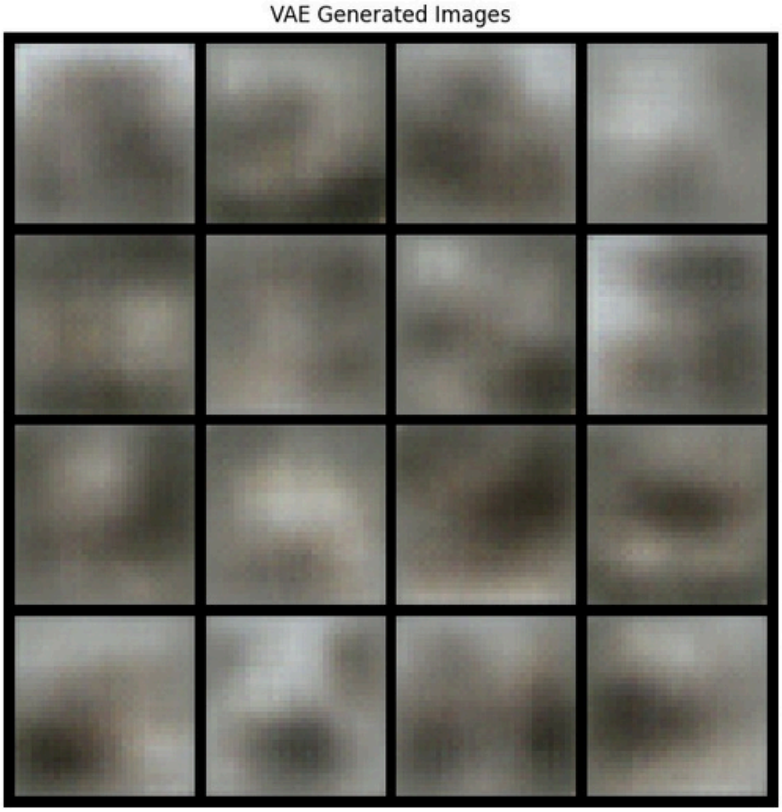
plt.title("VAE      Generated      Images")

plt.axis("off")

plt.show()
```

OUTPUT:

```
Using device: cpu
100%|██████████| 170M/170M [25:46<00:00, 110kB/s]
Dataset loaded successfully!
Epoch 1/5 - Loss: 184.7215
Epoch 2/5 - Loss: 141.4300
Epoch 3/5 - Loss: 116.7931
Epoch 4/5 - Loss: 108.2221
Epoch 5/5 - Loss: 102.1244
```



RESULT:

The Variational Autoencoder was trained on 5,000 CIFAR-10 images for 5 epochs; the average loss decreased from 184.7215 to 102.1244. 16 new images were generated from random latent vectors, and because of the short training and small subset they are blurry.